CVEs: Profiling Software Vulnerabilities over the Years

Process Book

By: Abhinav Shaurya Agarwal, Arkishman Ghosh, Wesley Jeong, Robert Caleb Jones, Brian Yang

Table of Contents

Table of Contents 2
Team Info 3
Team Agreement 4
Project Proposal 5
Map 6
Storyboard 21
Sketch 22
Decide 30

Team Info

Team Name: Can Visualize Everything

Team Leader: Robert Caleb Jones, rjones402@gatech.edu

Team Members:

- Abhinav Shaurya Agarwal, abhinav.shaurya@gatech.edu
- Arkishman Ghosh, aghosh372@gatech.edu
- Wesley Jeong, wesley.jeong@gatech.edu
- Robert Caleb Jones, rjones402@gatech.edu
- Brian Yang, byang356@gatech.edu

Team Agreement

How will you communicate?

We will communicate through text message to arrange meetings, remind members of deadlines, and to discuss any other small matters relevant to the project. Any big decisions related to tasking or project scope will be discussed in meetings, with text messaging used as a backup in case a member cannot attend a meeting.

How/when will you meet?

We will meet in person for weekly meetings, Fridays 2:00pm-3:00pm. The meeting date may be changed due to extenuating circumstances, and the location may vary based on the week. Additional meetings may be held if more planning is required to meet a deadline. Meetings may also be held online as necessary.

How will you collaborate on implementation?

Task distribution will be handled by the team leader and supporting discussion by members. Members' preference on tasking will be prioritized. Tasks may vary between independent tasks to tasks that involve multiple members working together. Distribution will be handled during team meetings.

To facilitate collaboration, we will be using a GitHub repository to host code and a Google Drive to host documentation, research, and other miscellaneous files relevant to the project.

How will you deal with non-performing members?

Non-performing members will first be confronted directly in team meetings to understand what may be harming their performance. The team will first try to work with the non-performing member to see what strategies may improve their performance.

In the situation that a team member is non-responsive (evading team meetings, not responding to text messages, etc.) and effort has been made to reach out to the member, the situation may be escalated to the class teaching staff for further guidance.

Signatures

Abhinav Shaurya Agarwal, Arkishman Ghosh, Wesley Jeong, Robert Caleb Jones, Brian Yang

Project Proposal

Title: CVEs: Profiling Software Vulnerabilities Over the Years

Abstract

Computers are ubiquitous in the current era, and so is the software that they run. People rely on the programs computers run everyday, trusting that the programs will perform as expected. However, any programmer knows that code comes with its bugs that can result in small errors, entire crashes, or even malicious exploits. With the ubiquity of computing and the prevalence of bugs, a database of vulnerabilities known as Common Vulnerabilities and Exposures, or CVEs, was put in place by MITRE to warn software owners of their risk. Each CVE record comes with information about the severity of the vulnerability, the date it was published, as well as the software vendor, product, and version the vulnerability exists on. This database is supplemented with the Known Exploited Vulnerabilities Catalog, which tracks cases in which a CVE was actually exploited in the wild. Using these data sources, we seek to gain insight into how vulnerabilities have evolved over time through various metrics, such as the most frequent type of vulnerability, most commonly exploited vulnerability, which year has the biggest quantity of severe vulnerabilities, and more. By doing this, we can warn developers what they should test for and be most careful about when writing code in hopes that these vulnerabilities may be alleviated.

Potential Data Sources

- CVE Database: https://github.com/CVEProject/cvelistV5
    - From https://www.cve.org
- Known Exploited Vulnerabilities Catalog: https://www.cisa.gov/known-exploited-vulnerabilities-catalog
- National Vulnerability Database: https://nvd.nist.gov/

Map

Project Plan

I. Audience
   1. **Software Developers**
   2. Vulnerability Researchers
   3. Cybersecurity Enthusiast

II. Audience Description
   1. What do they know?
      - Software developers are well versed in programming languages, software architecture models, and working in agile development environments to get code out fast. They have a classical computer science background majority of the time and understand the fundamentals of data structures and algorithms. Oftentimes, they focus on developing software in a certain area, such as cloud computing, web development, or operating systems.
   2. What are their interests?
      - A member of this audience would be interested in producing functional software that provides customer satisfaction. Of course, this means that shipping out software with bugs or vulnerabilities would not be in their interest. Understanding common mistakes that lead to vulnerabilities when programming would therefore be important to them.
   3. What visualization literacy do they have?
      - While not classically trained in visualization, most software developers are likely to be from higher education, so a moderate visualization literacy is fair to assume.
   4. At what level of detail will you present information to them?
      - With the possibility of a wide range of experience, we plan to present a wide range of detail. This will give those without a lot of knowledge the ability to understand some of the big ideas, while also letting those who have experience take away more nuanced insights.

III. Audience Questions
   1. Given KEVs across the dataset, what are the most common types of vulnerabilities being exploited?
   2. What is the most common severity of CVE across the entire dataset?
   3. Which vendors hold the highest number of CVEs across the entire dataset?
   4. Given KEVs across the dataset, what are the most common vendors getting exploited?
   5. Is there a relationship between CVSS severity and whether CVE actually ends up in KEV?
   6. How has the number of published CVE and their severity changed over the past year?

7. Which type of mistakes(CWE) gets exploited the most, compared to how often they show up in the data?
8. For the top 10 vendor/products with the highest number of unique CVEs from 2022-2026, what is the breakdown of their known CWE classifications by count and percentage?
9. What percentage of each product's CVEs lack a parsed CWE classification for top 10 vendor/products with the highest number of unique CVEs from 2022-2026?
10. How has the number of unique CVEs changed over time by CVSS attack vector?
11. Which vendors have the highest proportion of their CVEs appearing in the KEV catalog?
12. Which CWE categories tend to be associated with the highest-severity vulnerabilities?
13. Does the time between a CVE being reserved and published differ between vulnerabilities that are known to be exploited and those that are not?
14. Of the vendors with at least 100 CVEs, which vendors have the highest median CVSS score?
15. How does CVE severity distribution differ among the top 10 vendors by CVE count?
16. Which CWE categories experienced the largest increase in reported CVEs from 2022 to 2025?

IV. Data

1. CNA/Assigner
    - Categorical
    - CVE Numbering Authorities
        a) "CNAs are vendor, researcher, open source, CERT, hosted service, bug bounty provider, and consortium organizations authorized by the CVE Program to assign CVE IDs to vulnerabilities and publish CVE Records within their own specific scopes of coverage."
        b) https://www.cve.org/ProgramOrganization/CNAs
    - assignerOrgId refers to the ID used for the CNA, but there is no available mapping. We will have to create this ourselves
        a) assignerShortName can be used to help
2. CVE ID
    - Categorical
    - Unique ID for a CVE record
3. Date Published
    - Quantitative
    - The date the CVE record was published
4. Date Reserved
    - Quantitative
    - The date the CVE record was reserved

5. Date Rejected
    - Quantitative
    - The date the CVE record was rejected
6. State
    - Categorical
    - Details the state of the CVE, which is one of three possibilities
        a) Reserved: A CNA has reserved a CVE ID. This is the initial state of a CVE ID.
        b) Published: A CNA has populated the data associated with the CVE ID and published the CVE Record.
        c) Rejected: The CVE ID and the associated CVE Record should no longer be used. A Rejected CVE Record remains on the CVE List so that users know that the CVE ID and CVE Record are invalid.
        d) Cited from https://www.cve.org/ResourcesSupport/Glossary?activeTerm=glossaryRecord
7. CVSS Score
    - Quantitative
    - Score rating the severity of a vulnerability out of 10. Framework created by the Forum of Incident Response and Security Teams (FIRST) organization.
    - Not included on all records since was created after CVEs started being tracked
    - Can be set by a CNA or an Authorized Data Publisher (ADP)
        a) ADPs provide extra information about a CVE if the CNA has not
8. CVSS Severity
    - Ordinal
    - Can be one of four ratings
        a) Critical
            (1) Score of 9.0 - 10.0
        b) High
            (1) Score of 7.0 - 8.9
        c) Medium
            (1) Score of 4.0 - 6.9
        d) Low
            (1) Score of 0.1 - 3.9
        e) None
            (1) Score of 0.0
    - Corresponds to how soon a patch should be applied
9. Known Exploited Vulnerability (KEV)
    - Categorical
    - If the CVE is listed in CISA's KEV database, meaning it has been observed to be exploited in the wild

- We can go through the KEV database (https://www.cisa.gov/sites/default/files/feeds/known_exploited_vulnerabilities.json) to find an exact list of these
    a) The CVSS for these records however are listed in the CVE database

10. Product
    - Categorical
    - The product that features the CVE
11. Vendor
    - Categorical
    - The vendor of the product that has the CVE
12. Common Weakness Enumeration (CWE)/Problem Type
    - Categorical
    - Labels the CVE as a type of exploit
        a) Ex. remote code execution, SQL injection, etc
    - Not included in all CVE records
    - Some may not include the CWE ID

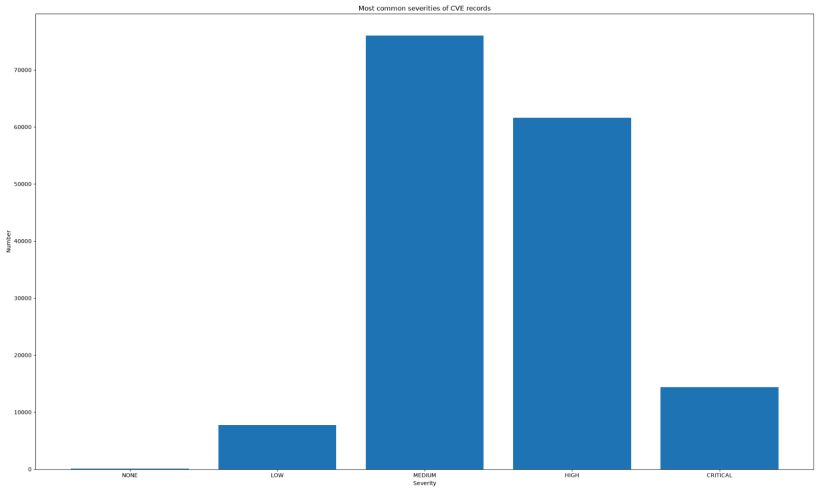
V. Data Cleanup Expectations

1. Earlier records may not have all expected data included, since many metrics surrounding CVEs were only developed after the CVE database was created. This isn't necessarily data we have to clean up, but it does mean we have less data to work with as we go back to earlier years. Some data that may not be listed in early CVE records can be found here: https://www.cve.org/About/RelatedEfforts
2. It's possible that to properly match CNA names to their org IDs, we will have to manually type out a mapping.
3. To find the CVSS of a KEV entry, we will have to refer back to the CVE record. This won't be too difficult since the CVE ID is included in the KEV entry.
4. The CVE records are not stored in one central area, meaning parsing and compiling data surrounding them is a bit difficult. We plan to find a way to compress the individual JSON files into another format.

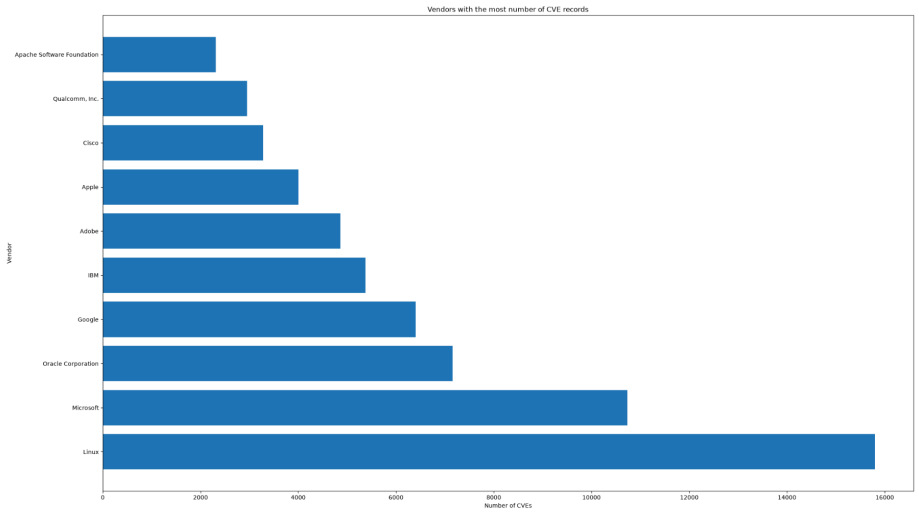
Data Exploration

Findings

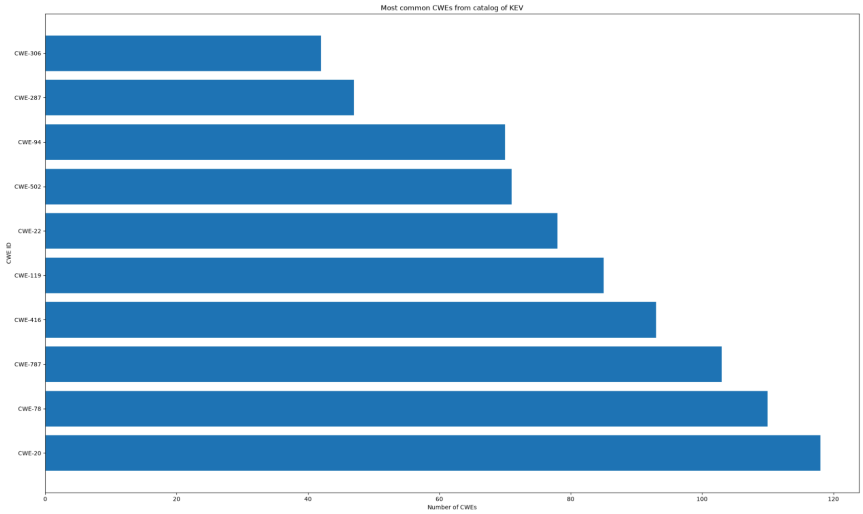
1. Robert Caleb Jones
    a. Out of all the CVE records, the most common severity is medium. This means the majority of reported CVEs are not extremely threatening if exploited; however, CVEs of high severity are a close second, so it is not safe to assume a vulnerability is of low impact.



    b. The results of vendors with the most CVEs are interesting. While at first it may appear as Linux having the most CVEs in total and therefore is the most insecure, it is important to note that it is an open source software; this means that vulnerabilities are easier to find since source code is provided and people who are contributing are encouraged to report bugs. If we contrast this with a private company with closed source software, reporting a vulnerability may be seen as a bad thing, since it could impact shareholder value. Interestingly, Microsoft has the second most CVEs. A comparison that could be made is that both Linux and Microsoft are vendors maintaining popular operating systems, and with the complexity of operating systems, it could be easier to find vulnerabilities in them. Further investigation on the vendors in the top 10 may be interesting to look through to see if a statement could be made about the number of vulnerabilities existing in code versus the number that actually get reported is more for Linux due to its open source nature.

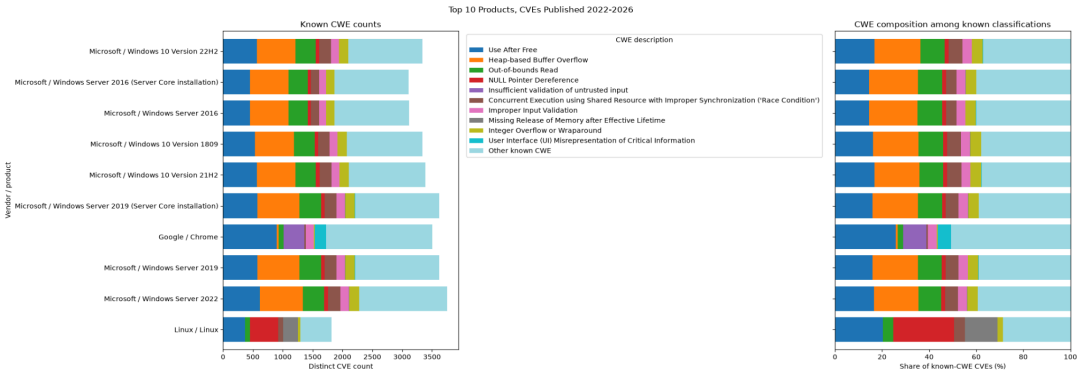


c. The overall most common type of vulnerability is CWE-20, or improper input validation. This makes sense, as to defend against user input, you would have to account for essentially infinite possibilities. Additionally, this is an umbrella CWE that encompasses many more specific types. For example, also in this top 10 is CWE-78 (OS Command Injection), CWE-22 (Path Traversal), CWE-502 (Deserialization of Untrusted Data), and CWE-94 (Code Injection). These are all situations that take in user input and fail to sanitize it. Another common category in this top 10 is memory management with CWE-787 (Out-of-bounds Write), CWE-416 (Use After Free), and CWE-119 (Improper Restriction of Operations within the Bounds of a Memory Buffer). These all have to do with low level memory management, indicating that this is something hard to get right. Finally, the last two CWEs in this list are CWE-287 (Improper Authentication) and CWE-306 (Missing Authentication for Critical Function), showing lack of user authentication is also common. These insights are very important to note, since this can directly prove the most common mistakes developers make when programming and gives indication of an overarching problem to fix.



2. Arkishman Ghosh
    a. For the top 10 vendor/products with the highest number of unique CVEs from 2022-2026, what is the breakdown of their known CWE classifications by count and percentage?

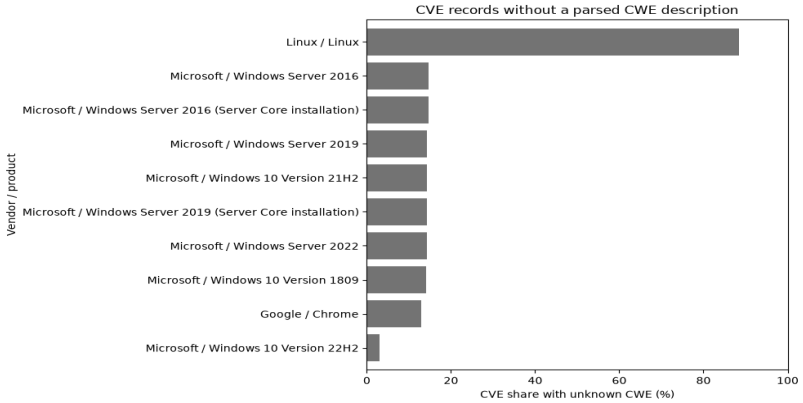
    Windows Server 2022 has the largest known-CWE volume among the selected products. Chrome's known classifications are relatively concentrated in Use After Free, while Windows products show a broader mix including Heap-based Buffer Overflow, Out-of-bounds Read, Improper Input Validation, and related categories. Linux falls under a different category. It has the highest total CVE count, but only about 11.6% of its CVEs have a known CWE classification. Therefore, its raw CWE counts and percentages describe only a small subset of its total CVEs. These graphs cannot support the conclusion that Linux has fewer weaknesses overall.



    b. What percentage of each product's CVEs lack a parsed CWE classification for top 10 vendor/products with the highest number of unique CVEs from 2022-2026?

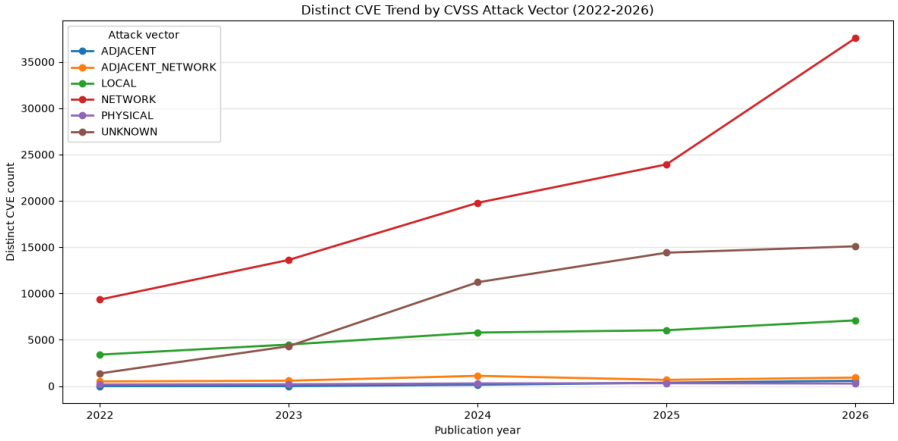
Linux: approximately 88.4% unknown
Most Windows products: approximately 3-15% unknown
Chrome: approximately 12.9% unknown

The important finding is not that Linux has fewer CWE categories. Rather, Linux's CWE data is much less complete in this dataset. Comparisons involving Linux should therefore be treated as lower-confidence than comparisons involving products with better CWE coverage. This graph describes metadata completeness, not vulnerability severity or risk.



c. How has the number of unique CVEs changed over time by CVSS attack vector?

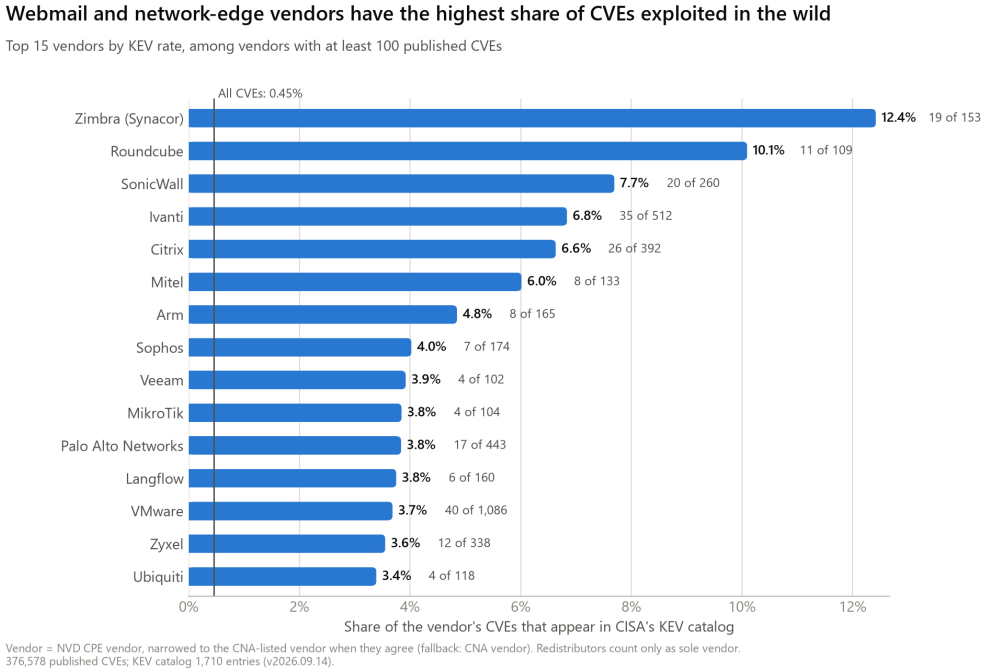
UNKNOWN increases from 1364 in 2022 to 14398 in 2025. This indicates substantially more missing attack-vector information in later years. Most newly published CVEs in this period are network-reachable, suggesting that remotely exploitable attack surfaces remain the dominant vulnerability exposure. Local vulnerabilities increased in raw count but declined proportionally. The strongest insight is that network-based exposure dominates the dataset, but the data does not prove that network-based vulnerabilities or real-world attacks are becoming more prevalent over time.



3. Abhinav Shaurya Agarwal

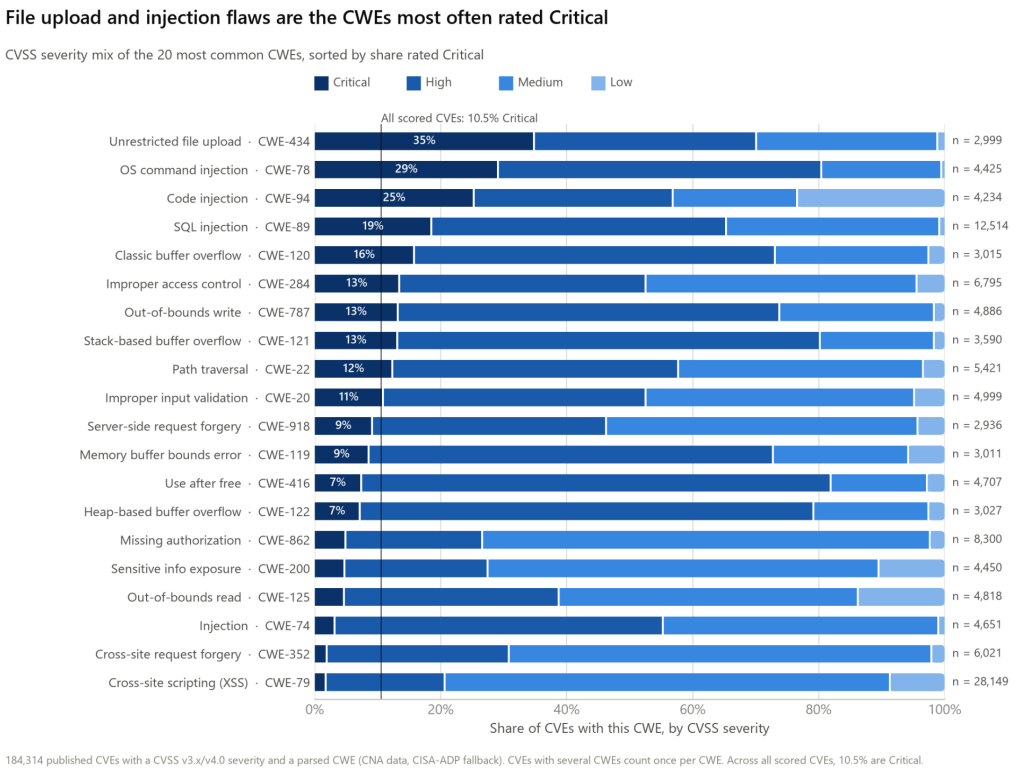
   a. Which vendors have the highest proportion of their CVEs appearing in the KEV catalog?

      About 1 in 8 Zimbra CVEs is exploited, compared with about 1 in 220 overall. Roundcube, SonicWall, Ivanti and Citrix follow. These are webmail servers, VPNs and firewalls, all of them internet-facing systems at the network edge that hold credentials.



   b. Which CWE categories tend to be associated with the highest-severity vulnerabilities?

      Unrestricted file upload (35% Critical), OS command injection (29%), code injection (25%) and SQL injection (19%) lead. What they have in common is that attacker input gets executed, often remotely and without logging in, which pushes the CVSS score to its maximum.



c. Does the time between a CVE being reserved and published differ between vulnerabilities that are known to be exploited and those that are not?

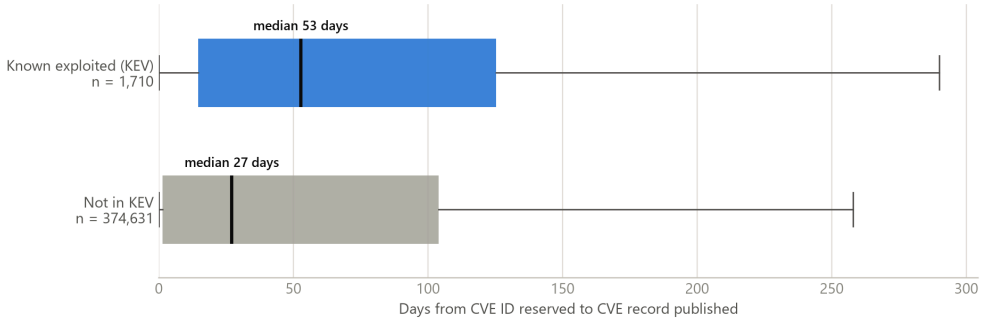
   Besides the medians, only 9% of KEV CVEs are published within a day of reservation, compared with 21% of the rest.

   KEV CVEs mostly come from major vendors (Microsoft alone is 22% of them). Those vendors reserve an ID early, then keep the CVE private until a patch ships, for example on Patch Tuesday. Most same-day publications come from:

   i. MITRE's older records;
   ii. high-volume CNAs like VulDB and Patchstack, which publish right away;
   iii. the Linux kernel CNA, which assigns CVEs after the fix is already public.

Exploited CVEs wait about twice as long between reservation and publication

Distribution of reserved-to-published delay; box = middle 50%, whiskers = 1.5 × IQR

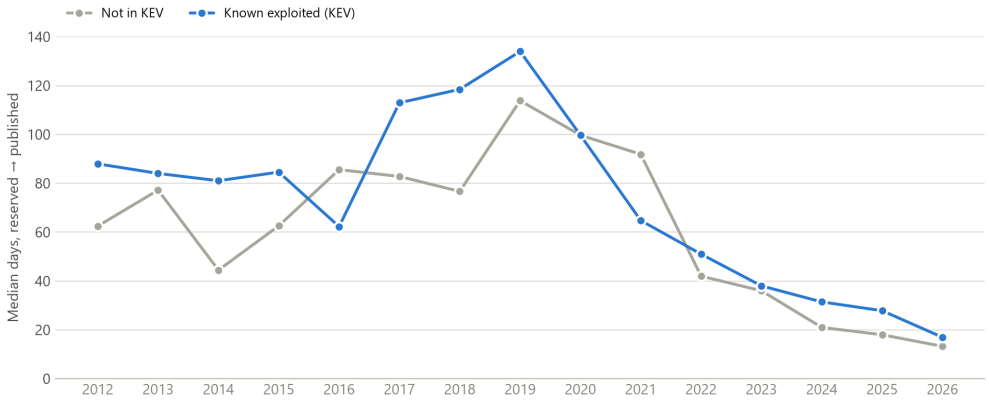


Published within a day of reservation: 9% of KEV vs 21% of other CVEs. Outliers beyond the whiskers are not drawn (max 5,220 days). 237 records dated before their reservation were dropped.

In the graph below, the blue line is KEV and the gray line is everything else. Each point is that year's median delay, and every year shown has at least 20 KEV CVEs, so the points aren't noisy. Blue is above gray in most years, and both lines drop sharply after 2021 as CNAs publish faster

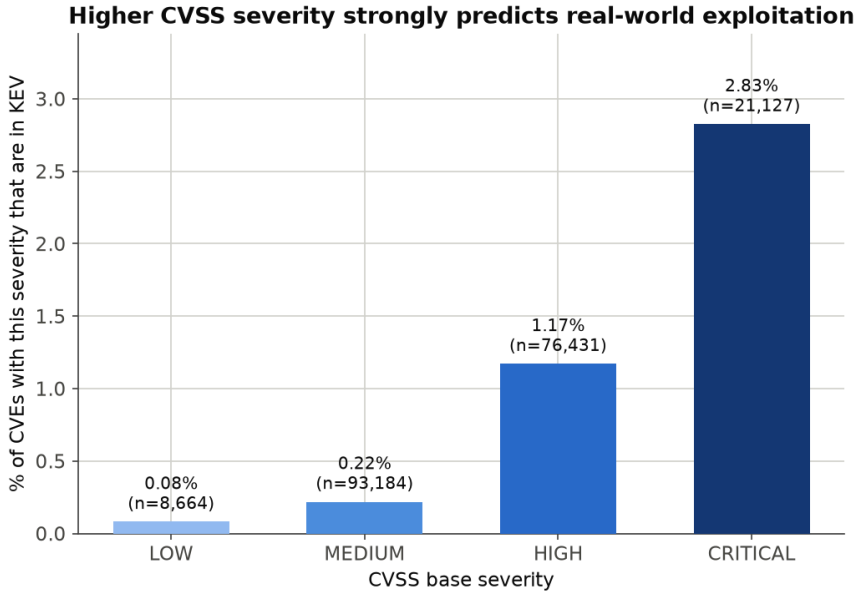
In 12 of 15 years, exploited CVEs waited longer before publication

Median reserved-to-published delay by publication year, 2012–2026 (every year has ≥ 20 KEV CVEs)



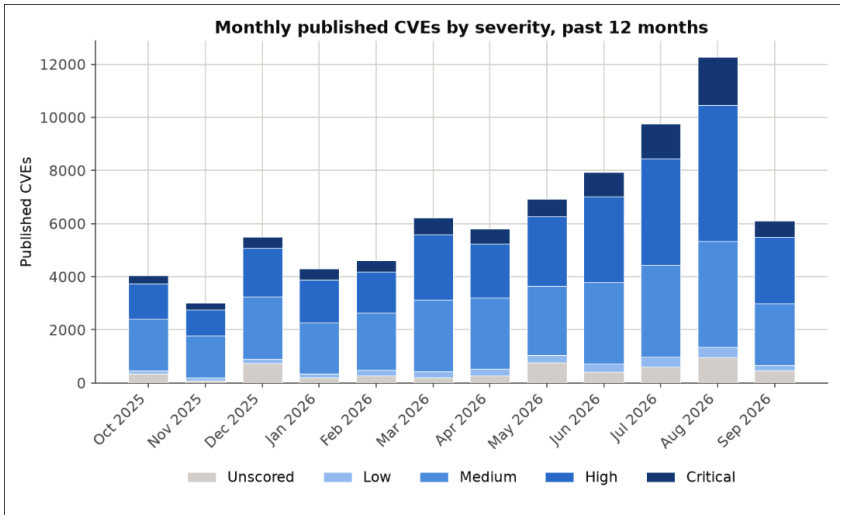
Brian Yang

a. Is there a relationship between CVSS severity and whether CVE actually ends up in KEV?



We can see that there is a positive relation between the severity level and the exploitation level of the CVE. The critical level of severity has 2.83% of CVEs that are in the KEV which indicates that higher severity means it is more likely to have accurate measuring.

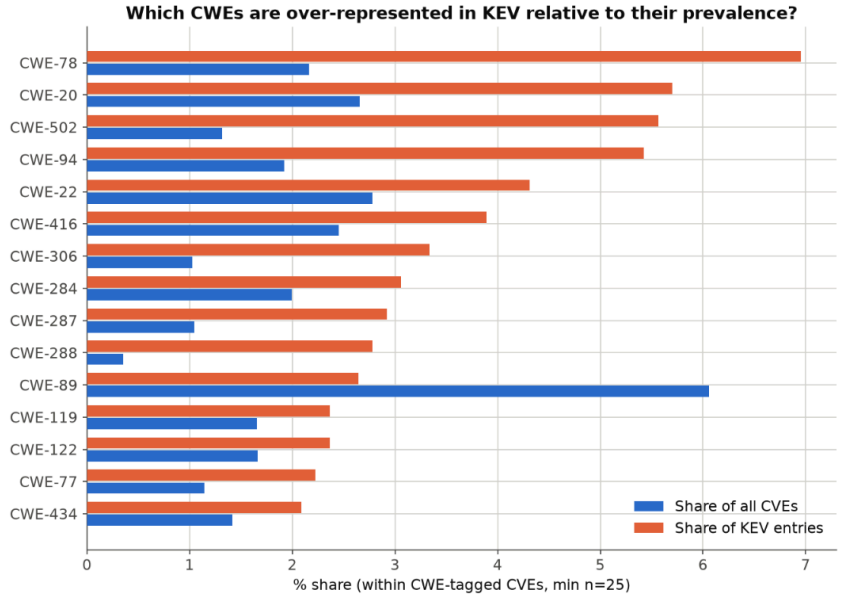
b. How has the number of published CVE and their severity changed over the past year?



The number of published CVE has tripled over the years from 4000 to now 12000. However September 2026 has a slight decline but this might be due to the fact that not the full data was pulled since we are still in the middle of the

month. Each severity has increased by about the same ratio across the years as well.

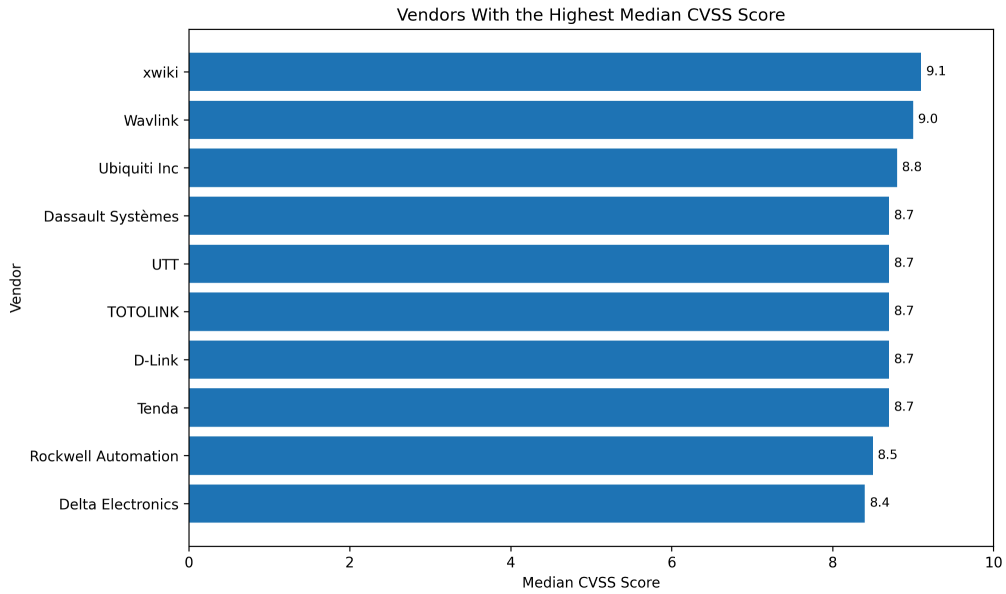
c. Which type of mistakes(CWE) gets exploited the most, compared to how often they show up in the data?



Comparing all the CWE, we see that CWE-78 gets exploited the most while CWE-89 represents the opposite pattern. Even though it has the highest CVE, it has one of the lowest share of KEV entries.

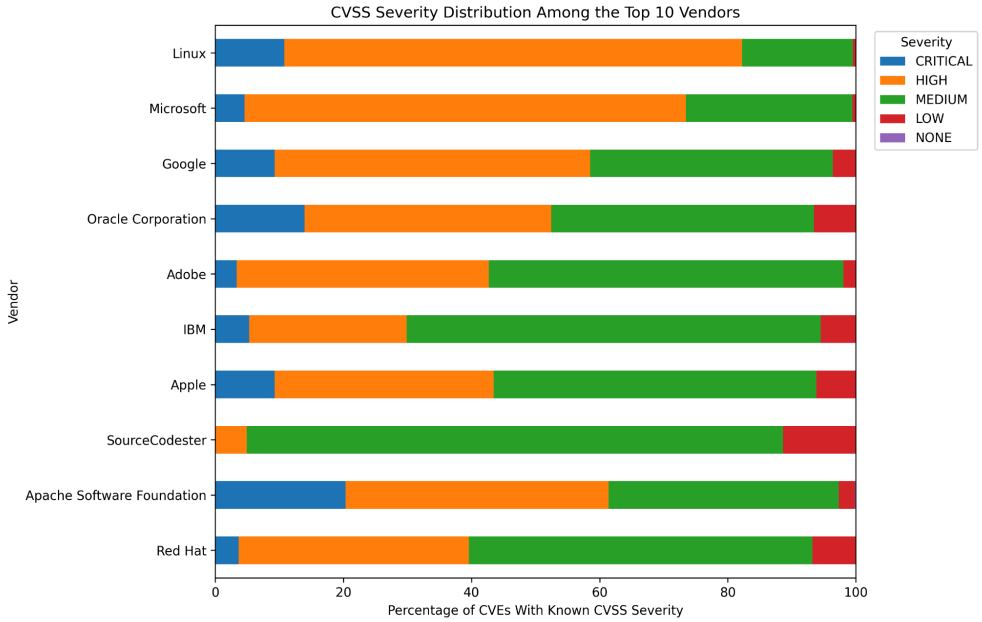
Wesley Jeong

a. Of the vendors with at least 100 CVEs, which vendors have the highest median CVSS score?



xwiki had the highest median CVSS score at 9.1, with Wavlink closely second at 9.0. These scores are considered critical, so this suggests that their reported vulnerabilities tend to be severe compared to other high-volume vendors.

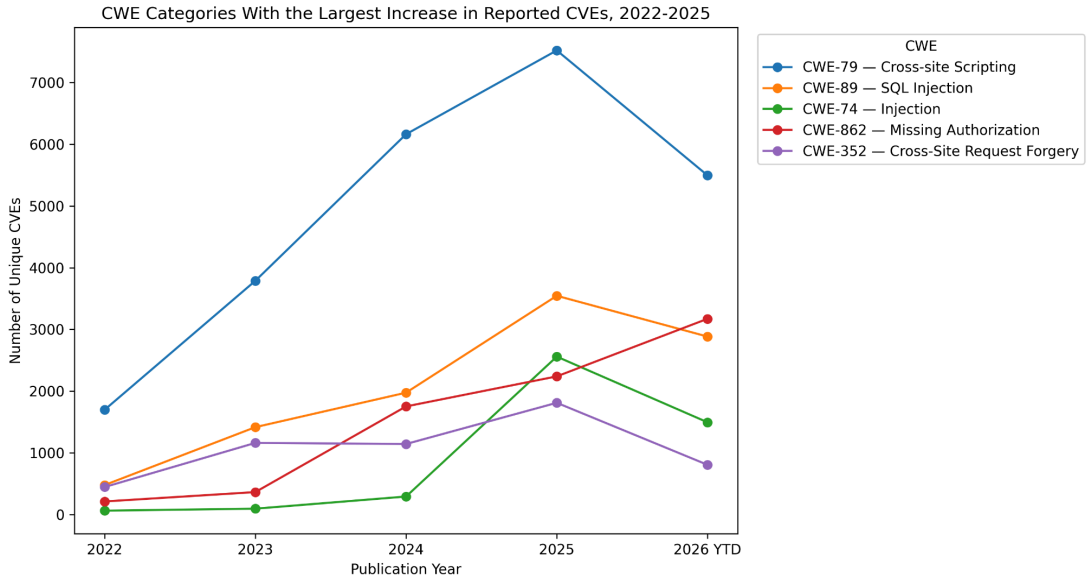
b. How does CVE severity distribution differ among the top 10 vendors by CVE count?



The top vendors differ significantly based on the types of severity ratings their CVEs receive. At the top, Linux and Microsoft have a much larger share of High-severity vulnerabilities, while vendors like IBM and SourceCodester have a

larger Medium-severity proportion. This goes to show that not all vendors with high CVE count have proportionally similar severities.

c. Which CWE categories experienced the largest increase in reported CVEs from 2022 to 2025?



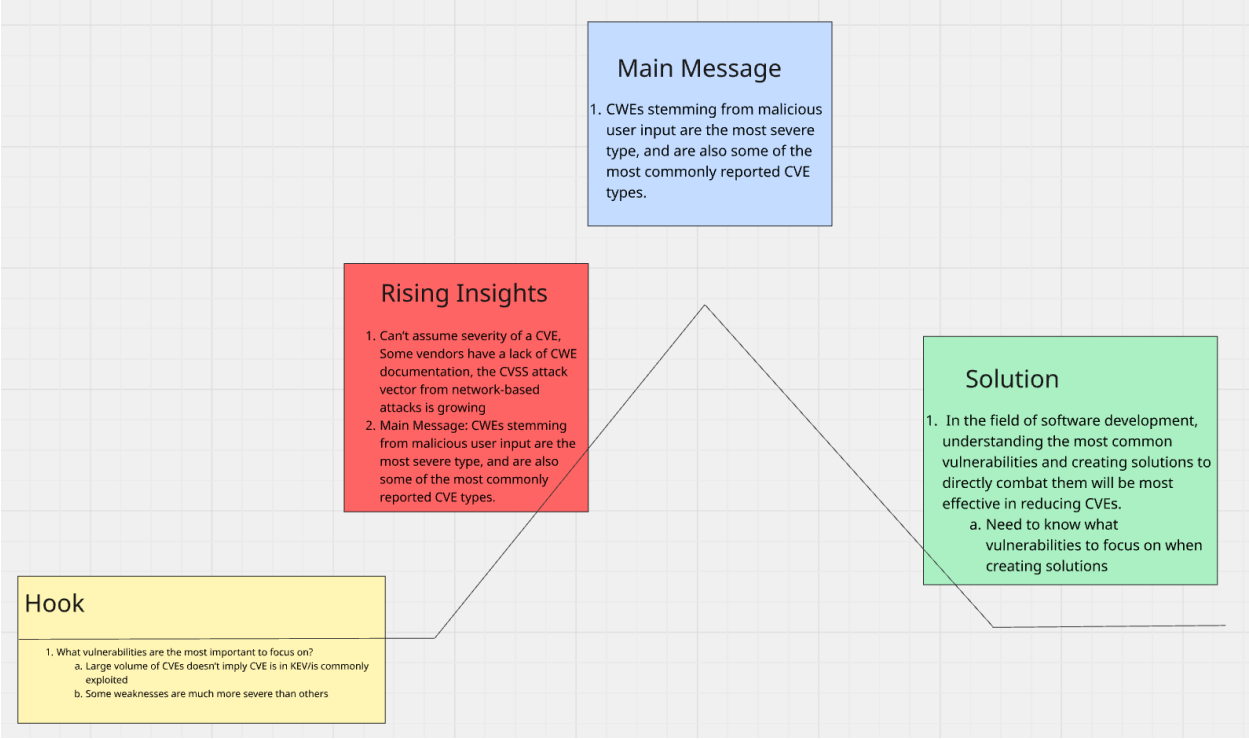
CWE-79: Cross-site Scripting is shown to be the most significant rise in the number of CVEs reported between 2022 and the end of 2025. The rise in number was from 1700 to 7500 instances. Some other types of security threats like SQL injection and missing authorization have also risen, indicating that web and authorization vulnerabilities increased.

Storyboard

Main Insight

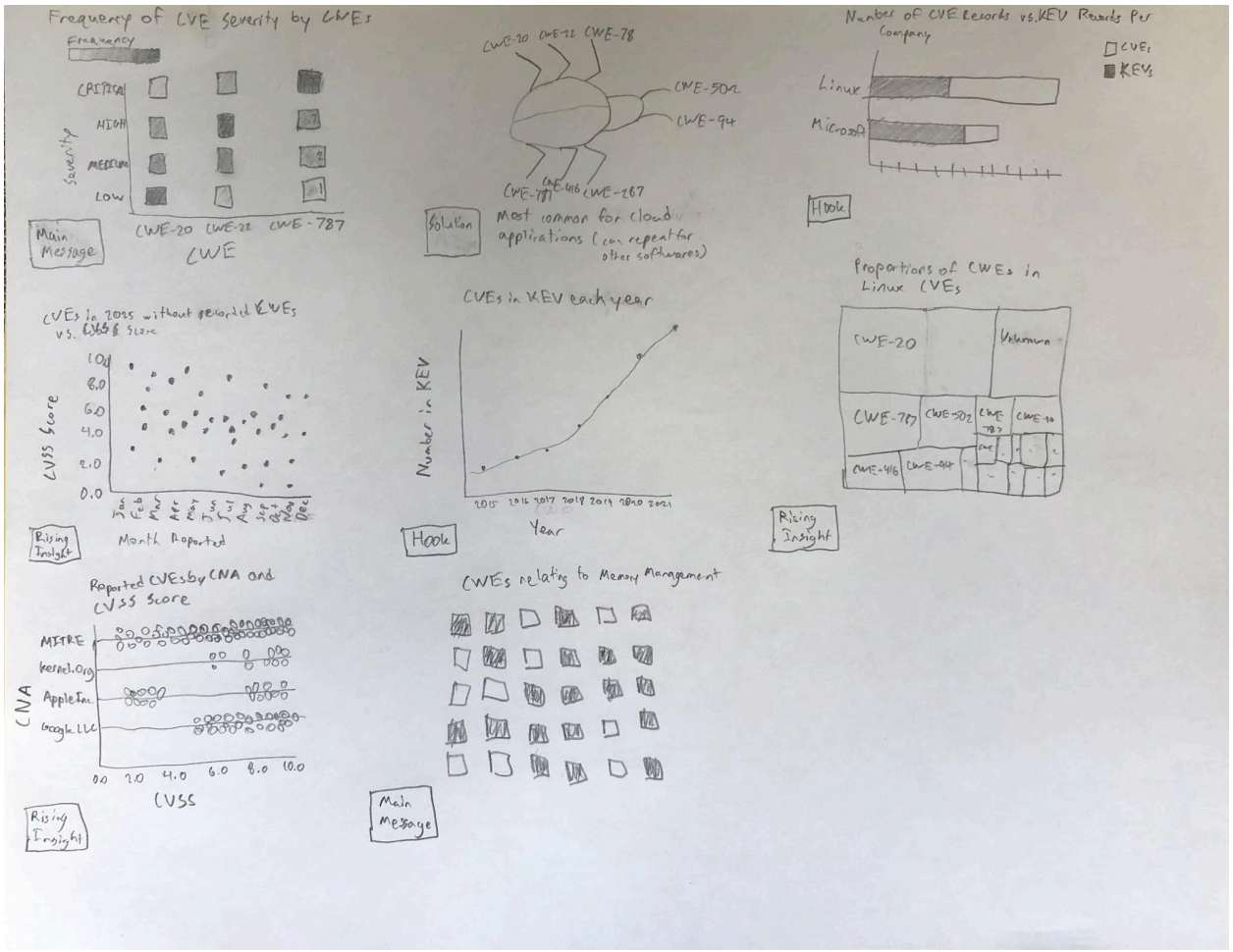
- Given CWEs of CVEs, identify which vulnerabilities are most common and which are most severe. Using this data, we can prioritize which weaknesses to test against and which vulnerability to expect when developing the code.
    - We chose this as our main insight because developers need to produce code that is secure and doesn't compromise the user experience. Looking for the most common, most severe CVEs helps us model what developers should test against and will help reduce the number of CVEs in the future.

Story Arc

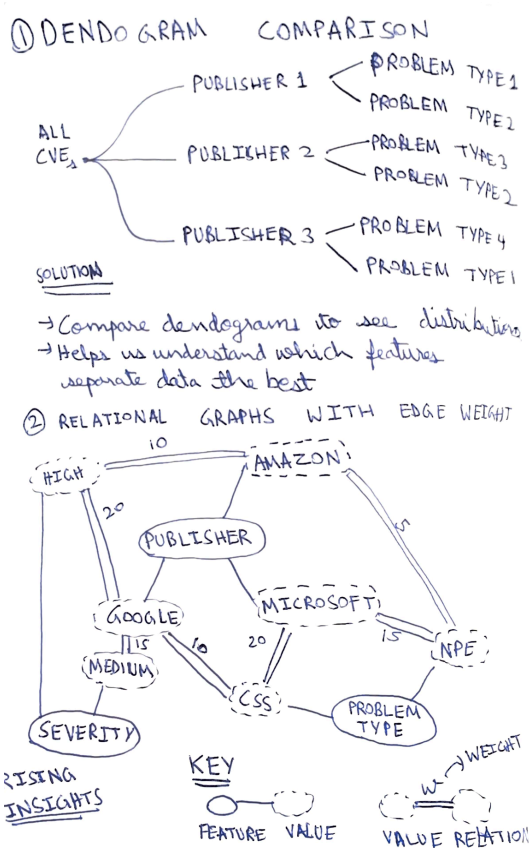


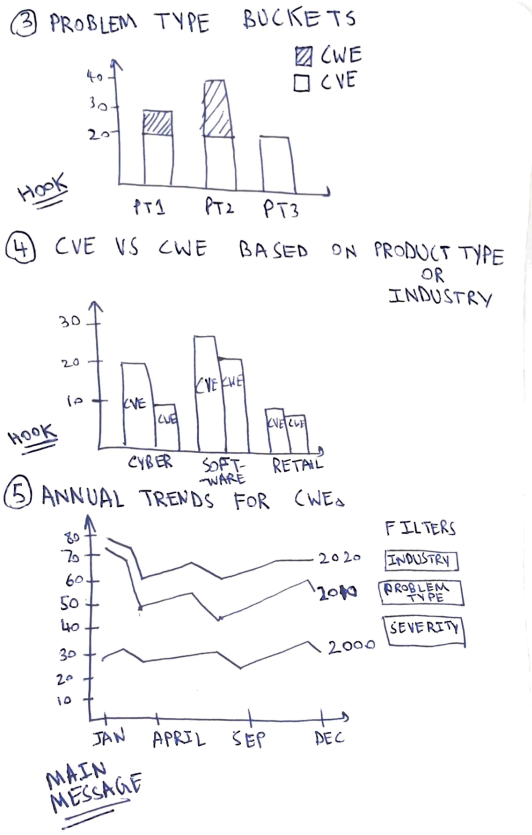
Sketch

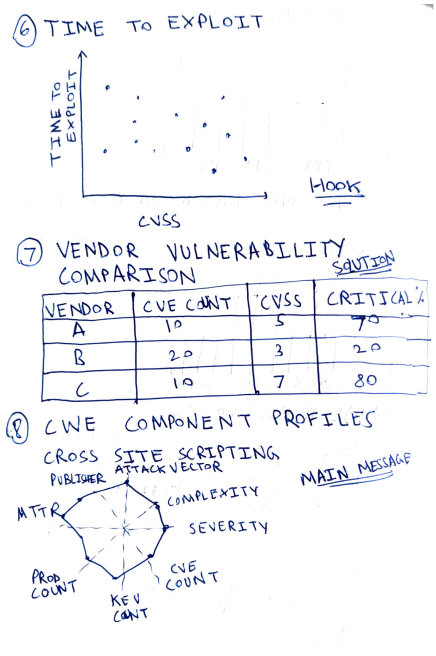
1. Robert Caleb Jones (1-8)



2. Arkishman Ghosh (9-16)



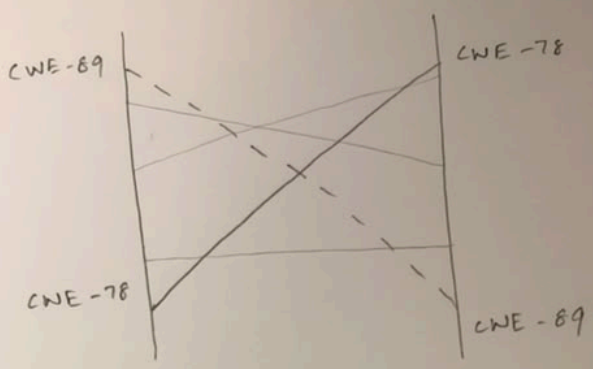




3. Abhinav Shaurya Agarwal (17-24)

ABHINAV SHAURYA AGARWAL

1) Hook



line up = exploited more than reported

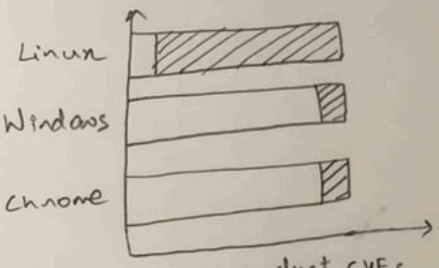
— thick = story CWEs
— gray = others

2) Rising Insight



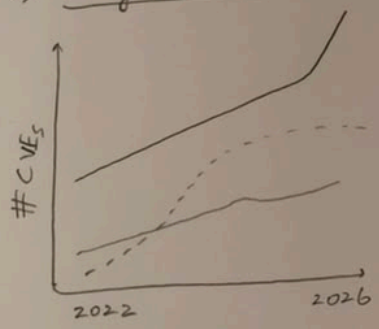
bar = CVEs published / month
shaded = high + critical

3) Rising Insight



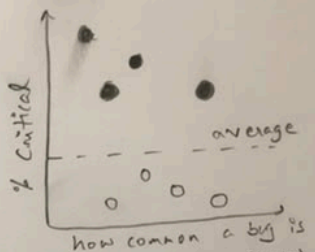
shaded = CVE with no CWE
white = CVE with CWE

4) Rising Insight



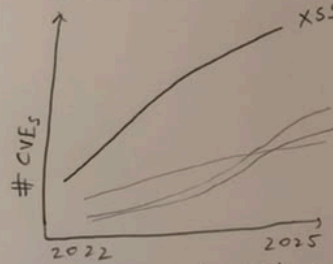
dark = network
light = local
dash = unknown attack

5) Main Message

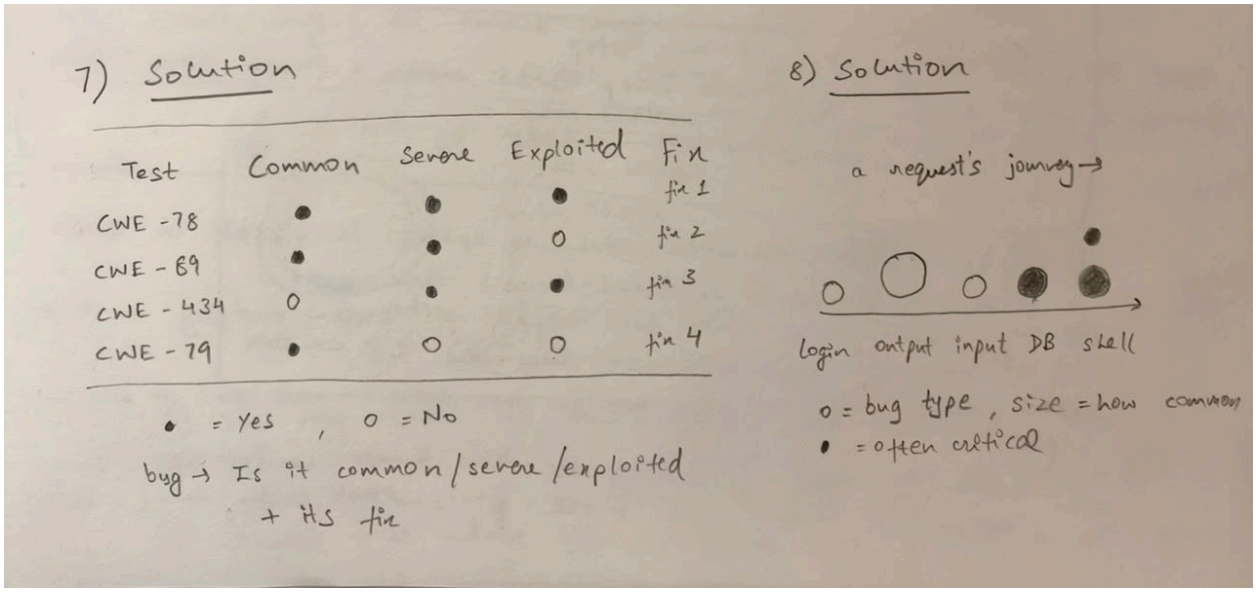


o = bug type (CWE)
• = above average

6) Main Message

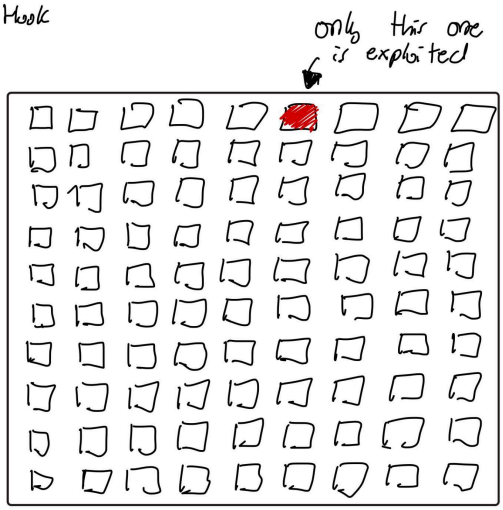


— = XSS (fastest growing)
— = other rising CWEs



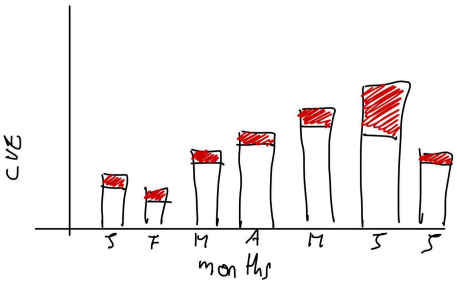
4. Brian Yang (25-32)

1) Hook



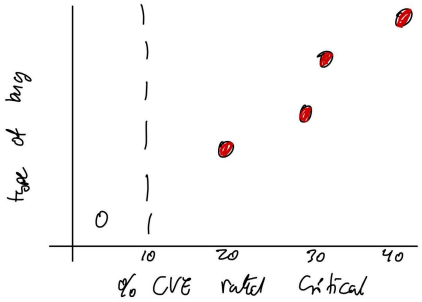
- CVE that are in KEV

2) Hook



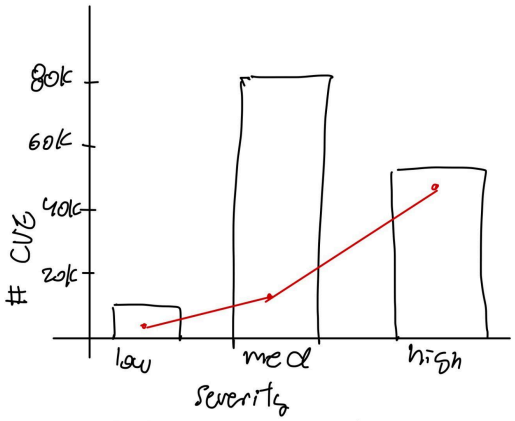
bar - how many CVE published per month
Red - Critical CVE

3) Hook



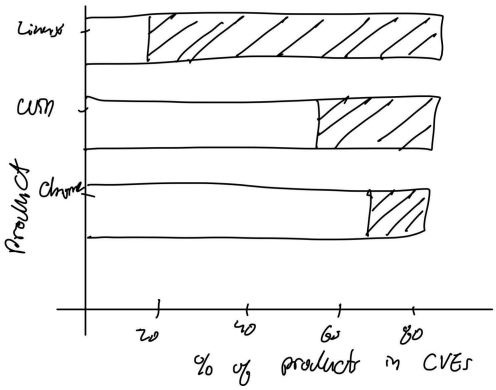
Red - above average
dash line - average
grey - below average

4) Rising Insight



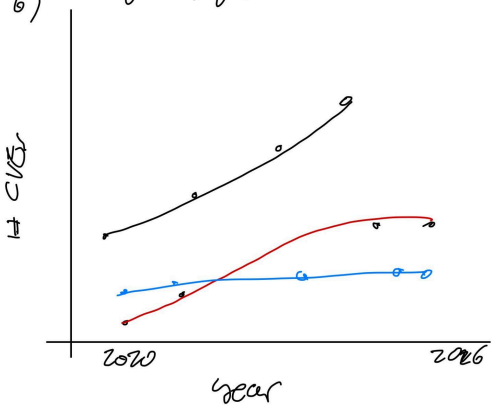
Red - Actual CVE in KEV

5) Rising Insight



stripe - CVE with no CWE label
white - CVE with CWE Label

6) Rising Insight



black - network CVE
red - unknown attack method
blue - local CVE

7) main message



O - bug type

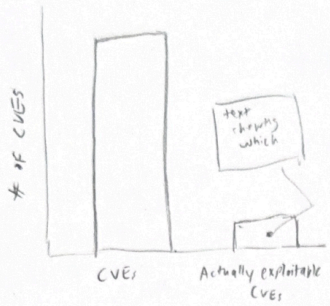
8) Solution

Test	common	severe	Exploited	Fix
CVE-1	◎	◎	◎	fix 1
CVE-2	◎	O	◎	fix 2
CVE-3	O	◎	◎	fix 3

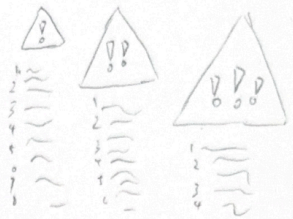
Each bug has => common / severe/exploited & its fixes

5. Wesley Jeong (33-40)

Hook



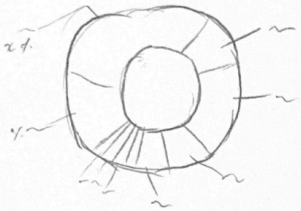
Hook



List of CVEs by severity

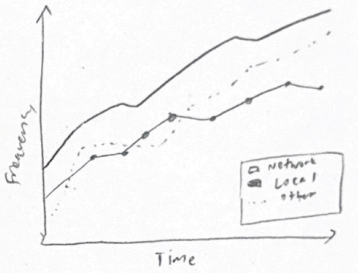
Rising Insight

Recorded CVEs in top X Vendors



Rising Insight

Network vs Local Attack Vendors



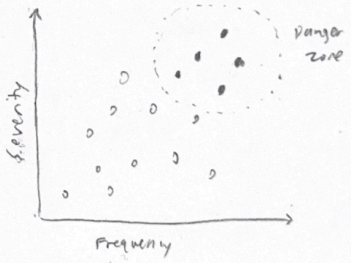
Rising Insight

Most common CWEs

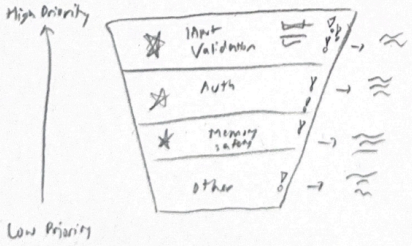
▨ = input-related



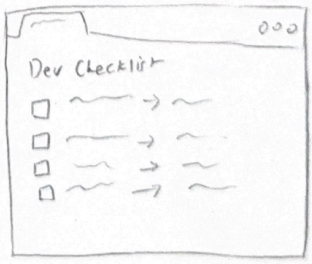
Main Message



Main Message



Solution



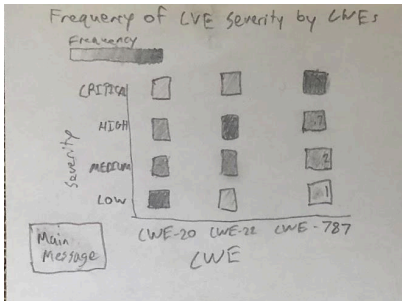
Decide

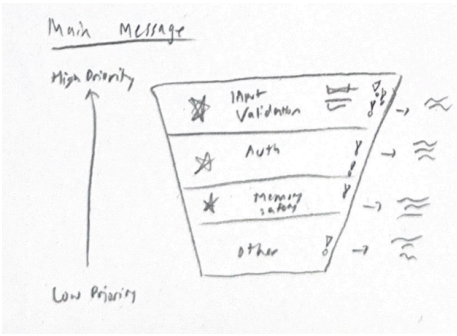
Affinity Diagramming

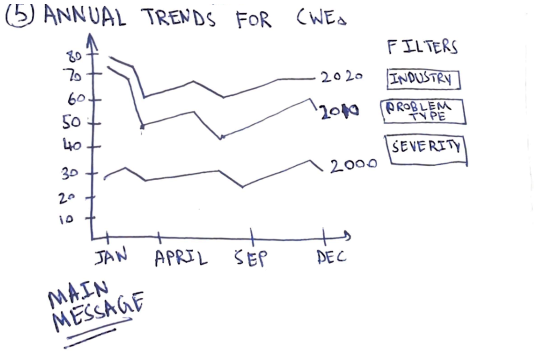
Sketch ID	Message ID	Author	Votes
1, 25	3, 1	RCJ, BY	WJ, RCJ, ASA, BY
2	4	RCJ	WJ
3, 11, 29	1, 5	RCJ, AG, BY	RCJ, BY
4, 38	2	RCJ, WJ	
5	1	RCJ	
6	2	RCJ	AG
7	2	RCJ	ASA
8	3	RCJ	
9	4	AG	
10	2	AG	
12	1	AG	
13	3	AG	AG, RCJ
14	1	AG	ASA
15	4	AG	
16	3	AG	WJ, AG, BY
17	1	ASA	
18, 26	2, 1	ASA, BY	
19	2	ASA	
20, 30	2	ASA, BY	
21, 27, 31	3, 1, 3	ASA, BY	RCJ, ASA, BY
22, 36	3	ASA, WJ	WJ
23, 32	4	ASA, BY	ASA
24	4	ASA	AG

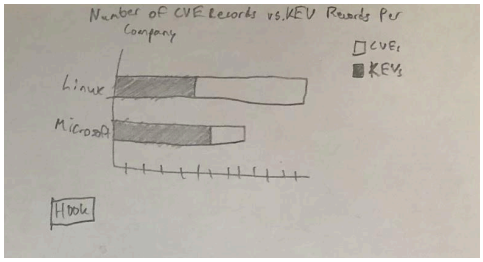
28	4	BY	
33	1	WJ	
34	1	WJ	
35	2	WJ	
37	3	WJ	
39	4	WJ	AG, RCJ, WJ, ASA, BY
40	4	WJ	

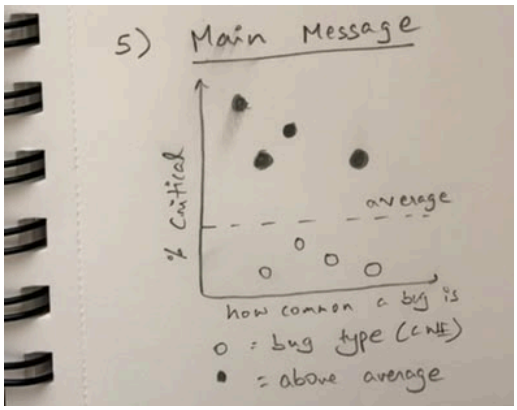
Voting Results

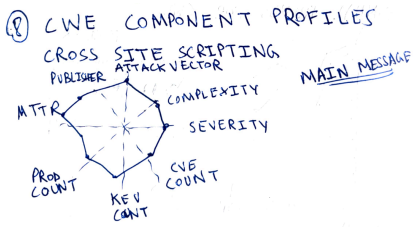












Decision Rationale

An audience member needs to understand the severity of the problem. For us, the best way to represent the impact is through visual elements like heatmaps. Per CWE, we want to represent how frequently a CWE comes up. Once the audience is hooked onto the impact of the problem, we explore different dimensions of the CVEs and CWEs: one of them being annual trends. We want the audience to be able to interpret seasonal patterns across different dimensions like product types, problem types etc. and compare them across multiple years. This gives the viewer the flexibility to focus on one problem at a time. Once the problem is selected, the funnel view allows the viewer to see how to break down the problem into layers. This could be related to the phases where the problem got introduced in the Software Development Lifecycle or product and publisher level breakdowns. An interesting footnote that gets introduced to the audience is “how confident are we about our data?” The comparative bar graph showing CVEs that are also KEVs helps us understand the relationship between how common a vulnerability is and how often it is actively exploited. Overall, these design choices help us move from a broad

overview of the vulnerability landscape to more specific patterns and possible areas of concern. The combination of overview visuals,comparisons, and drill-down views also lets different audience members explore the data at the level of detail that is more useful to them.

Prototype Phase 1

Clean Dataset [Caleb] Friday

[Insert link here]

Partly Implemented Visualization [Abhinav] Saturday

- Tableau has a way to embed visualizations in websites?
  [Insert screenshots here]

Drafts for More Visualizations [Brian & Arkishman] Saturday

- Have an example done in another software not embedded on site

Layout Design [Wesley] Saturday

[Insert screenshots here]

Prototype Phase 2

Polished Visualization [Abhinav] Saturday

Partly Implemented Visualizations [Brian & Arkishman] Saturday

Further Drafts [Caleb]

Interactions & Storytelling [Wesley] Sunday

- Does Tableau interactions get embedded? Probably generate, use d3 JS library